

UNIVERSIDADE TECNOLÓGICA FEDERAL DO PARANÁ

ÁLISON CHRISTIAN REBOUÇAS VIDAL DE CARVALHO

**COMPARAÇÃO PRÁTICA ENTRE MODELAGEM RELACIONAL E MODELAGEM
EM GRAFOS PARA CONSULTAS DE RECOMENDAÇÃO EM E-COMMERCE**

CORNÉLIO PROCÓPIO

2026

ÁLISON CHRISTIAN REBOUÇAS VIDAL DE CARVALHO

**COMPARAÇÃO PRÁTICA ENTRE MODELAGEM RELACIONAL E MODELAGEM
EM GRAFOS PARA CONSULTAS DE RECOMENDAÇÃO EM E-COMMERCE**

**A Practical Comparison Between Relational and Graph Data Modeling for
Recommendation Queries in E-commerce**

Trabalho de Conclusão de Curso apresentado ao
Curso de Engenharia de Software da Universidade
Tecnológica Federal do Paraná, como requisito
parcial para a obtenção do título de Bacharel.
Orientador: Prof. Dr. Eduardo Cotrin Teixeira.

CORNÉLIO PROCÓPIO

2026



Esta licença permite download e compartilhamento do trabalho desde que sejam atribuídos créditos ao(s) autor(es), sem a possibilidade de alterá-lo ou utilizá-lo para fins comerciais. Conteúdos elaborados por terceiros, citados e referenciados nesta obra não são cobertos pela licença.

AGRADECIMENTOS

Agradeço aos meus pais, Senio Vidal de Carvalho e Ana Cristina Rebouças Vidal de Carvalho, pelo apoio incondicional, tanto financeiro quanto psicológico, que me proporcionou a base necessária para concluir esta etapa da minha formação.

Aos meus amigos, pelo incentivo constante e pela presença nos momentos mais desafiadores.

A todos os professores do curso de Engenharia de Software, pela dedicação, compromisso e por todo o conhecimento compartilhado ao longo desta jornada.

Ao meu orientador, Prof. Dr. Eduardo Cotrin Teixeira, pela paciência, disponibilidade e pelas valiosas orientações que foram fundamentais para o desenvolvimento deste trabalho.

RESUMO

O crescimento exponencial do comércio eletrônico tem impulsionado a necessidade de sistemas de recomendação mais eficientes e escaláveis. Este trabalho apresentou uma comparação prática de desempenho e expressividade entre bancos de dados relacionais e bancos de dados orientados a grafos no contexto de recomendações para e-commerce. Utilizou-se o conjunto de dados público da Olist, modelado no PostgreSQL (com indexação B-Tree) e no Neo4j (Grafo de Propriedades Rotuladas), submetidos a um *benchmark* automatizado com amostragem dinâmica para evitar viés de memória *cache*. O experimento testou desde filtragens colaborativas clássicas até algoritmos complexos de menor caminho e testes de estresse com múltiplas junções. Os resultados demonstraram um claro ponto de inflexão arquitetural: o PostgreSQL foi superior em consultas de baixa profundidade, resolvendo análises de cesta de compras em médias de 0,35 ms contra 2,90 ms do Neo4j. Contudo, ao submeter os motores a um estresse de rede estendida (4 saltos lógicos), o PostgreSQL entrou em colapso combinatório, resultando em *timeout* por esgotamento de processamento, enquanto o Neo4j sobreviveu à travessia com tempos consistentes. Na busca por caminhos mais curtos, o banco de grafos foi mais de 160 vezes mais rápido. Conclui-se que, nos cenários avaliados, o modelo relacional mostrou-se mais eficiente para recomendações rasas em bases de tamanho moderado, enquanto a adoção de grafos torna-se especialmente adequada para a descoberta de redes ocultas e para a manutenção de latência estável em consultas de alta conectividade.

Palavras-chave: Banco de Dados; Grafos; Neo4j; PostgreSQL; Sistemas de Recomendação; E-commerce.

ABSTRACT

The exponential growth of e-commerce has driven the need for more efficient and scalable recommendation systems. This work presented a practical comparison of performance and expressiveness between relational databases and graph-oriented databases in the context of e-commerce recommendations. The Olist public dataset was used, modeled in PostgreSQL (with B-Tree indexing) and Neo4j (Labeled Property Graph), subjected to an automated benchmark with dynamic sampling to avoid memory cache bias. The experiment tested everything from classic collaborative filtering to complex shortest-path algorithms and stress tests with multiple joins. The results demonstrated a clear architectural inflection point: PostgreSQL was superior in low-depth queries, solving market basket analysis in averages of 0.35 ms versus 2.90 ms for Neo4j. However, when subjecting the engines to an extended network stress (4 logical hops), PostgreSQL entered a combinatorial collapse, resulting in a timeout due to processing exhaustion, while Neo4j survived the traversal with consistent times. In the search for shortest paths, the graph database was over 160 times faster. It is concluded that, in the evaluated scenarios, the relational model proved more efficient for shallow recommendations in moderately sized datasets, while graph adoption becomes especially suitable for discovering hidden networks and maintaining stable latency in highly connected queries.

Keywords: Databases; Graphs; Neo4j; PostgreSQL; Recommendation Systems; E-commerce.

LISTA DE FIGURAS

Figura 1: Fluxograma metodológico dos objetivos específicos	4
Figura 2 - Mecanismo de Junção (Join) e Busca por Índice B-Tree no Modelo Relacional	10
Figura 3 - Estrutura Conceitual de um Grafo de Propriedades Rotuladas (LPG)	13
Figura 4 - Matriz de Interação e Mecanismo de Dedução por Filtragem Colaborativa.....	16
Figura 5 - Estrutura Relacional Simplificada do Dataset Olist	26
Figura 6 Arquitetura Geral e Fluxo de Execução do Experimento.....	28
Figura 7 - Diagrama Entidade-Relacionamento do Protótipo A (Relacional)	30
Figura 8 - Modelo de grafo do Protótipo B (Neo4j).....	31

LISTA DE TABELAS

Tabela 1 Tempo de Resposta (ms) para Consultas de Baixa Profundidade	34
Tabela 2 - Escalonamento de Profundidade e Degradação de Performance (ms)	35
Tabela 3 - Tempo de Resposta (ms) para Descoberta de Caminho (Q5).....	37

LISTA DE ABREVIATURAS E SIGLAS

3FN	Terceira Forma Normal
ACID	Atomicidade, Consistência, Isolamento e Durabilidade
BDOG	Banco de Dados Orientado a Grafos
CPU	Central Processing Unit
CSV	Comma-Separated Values
DER	Diagrama Entidade-Relacionamento
ETL	Extract, Transform, Load
FBC	Filtragem Baseada em Conteúdo
FC	Filtragem Colaborativa
FK	Foreign Key
GPU	Graphics Processing Unit
I/O	Input/Output
LOC	Lines of Code
LPG	Labeled Property Graph
NoSQL	Not Only SQL
NVMe	Non-Volatile Memory Express
RAM	Random Access Memory
RF	Requisito Funcional
RNF	Requisito Não Funcional
SGBD	Sistema Gerenciador de Banco de Dados
SGBDR	Sistema Gerenciador de Banco de Dados Relacional
SGBDOR	Sistema Gerenciador de Banco de Dados Objeto-Relacional
SQL	Structured Query Language
SR	Sistema de Recomendação
SSD	Solid-State Drive
TCO	Total Cost of Ownership
VS Code	Visual Studio Code

SUMÁRIO

1	INTRODUÇÃO	1
1.1	Problema de Pesquisa	2
1.2	Objetivos	3
1.2.1	Objetivo Geral.....	3
1.2.2	Objetivos Específicos	3
1.3	Justificativa.....	4
1.4	Organização do Trabalho	5
2	FUNDAMENTAÇÃO TEÓRICA	7
2.1	Banco de Dados	7
2.2	Banco de Dados Relacional	8
2.3	Banco de Dados NoSQL	10
2.4	Banco de Dados Orientado a Grafos	11
2.5	Sistemas de Recomendação	13
2.6	Filtragem Colaborativa.....	14
2.7	Filtragem Baseada em Conteúdo.....	17
2.8	Ferramentas Utilizadas	18
2.8.1	PostgreSQL.....	19
2.8.2	Neo4j e Cypher.....	19
2.8.3	Linguagem Python e Ecossistema de Dados	20
3	METODOLOGIA	22
3.1	Abordagem de Pesquisa.....	22
3.2	Ferramentas e Tecnologias	23
3.2.1	Ambiente de Execução.....	24
3.3	Objeto de Estudo.....	24
3.4	Arquitetura do Experimento	26
4	PROPOSTA DO SISTEMA	29
4.1	Modelagem de Dados.....	29
4.1.1	Protótipo A: Modelo Relacional (PostgreSQL)	29
4.1.2	Protótipo B: Modelo de Grafo (Neo4j)	30
4.2	Arquitetura do Motor de Benchmark e Captura de Discrepâncias ..	31
4.3	Protótipos das Consultas	32
5	ANÁLISE DOS RESULTADOS E DISCUSSÃO.....	34
5.1	Desempenho em Consultas de Baixa Profundidade.....	34

5.2	O Teste de Estresse e a Explosão Combinatória	35
5.3	A Descoberta de Caminhos (Shortest Path) e Expressividade	36
6	CONSIDERAÇÕES FINAIS	38
	REFERÊNCIAS.....	40

1 INTRODUÇÃO

A popularização da internet e o avanço contínuo das tecnologias da informação e comunicação transformaram radicalmente o comportamento de consumo global, consolidando o comércio eletrônico (*e-commerce*) como um dos pilares da economia contemporânea. Com o aumento expressivo no volume de transações e interações diárias, plataformas de vendas *online* passaram a gerar e armazenar uma quantidade massiva de dados complexos. Para converter essa vasta quantidade de informações em vantagens competitivas e em melhorias tangíveis na experiência do usuário, os Sistemas de Recomendação (SR) tornaram-se ferramentas de *software* indispensáveis. Esses algoritmos analisam o histórico de interações, os padrões de navegação e as preferências dos consumidores para sugerir itens relevantes em tempo real, aumentando significativamente as taxas de conversão e a fidelização de clientes.

Tradicionalmente, a persistência e o processamento dos dados que alimentam esses sistemas têm sido delegados aos Sistemas Gerenciadores de Bancos de Dados Relacionais (SGBDR), como o PostgreSQL. Baseados no modelo matemático introduzido por Edgar F. Codd na década de 1970, os bancos de dados relacionais oferecem garantias robustas de integridade estrutural e consistência transacional, suportadas pelas propriedades ACID (Atomicidade, Consistência, Isolamento e Durabilidade). No entanto, à medida que a arquitetura dos sistemas de recomendação evolui para explorar conexões cada vez mais profundas — cruzando dados de usuários, pedidos interligados, categorias e produtos —, o modelo relacional começa a esbarrar em limitações arquiteturais de desempenho. A necessidade de realizar múltiplas junções (*joins*) lógicas em tempo de execução para percorrer tabelas normalizadas resulta em um alto custo computacional e latência elevada, especialmente quando o volume de dados cresce exponencialmente.

Em resposta a essas limitações estruturais, o movimento NoSQL (*Not Only SQL*) ganhou força na indústria de tecnologia, introduzindo novos paradigmas de armazenamento desenhados para domínios específicos no ecossistema de dados. Dentro desse espectro, destacam-se os Bancos de Dados Orientados a Grafos (BDOG), exemplificados por tecnologias consolidadas como o Neo4j. Diferentemente do modelo relacional, a abordagem baseada em grafos trata os relacionamentos como entidades de primeira classe. Por meio do conceito de adjacência livre de índice

(*index-free adjacency*), os relacionamentos passam a ser armazenados como conexões explícitas entre entidades, permitindo que a travessia de dados altamente interconectados dependa principalmente do subgrafo efetivamente percorrido, e não da reconstrução sucessiva de junções sobre toda a base. É nesse cenário de transição tecnológica e busca por melhor desempenho em consultas altamente conectadas que a presente pesquisa foi conduzida.

1.1 Problema de Pesquisa

O projeto e a implementação de um sistema de recomendação eficiente exigem decisões arquiteturais precisas por parte dos engenheiros de software, sendo a escolha da tecnologia de persistência uma das mais críticas e impactantes no ciclo de vida do produto. Os projetistas frequentemente se deparam com o dilema técnico de utilizar tecnologias relacionais maduras, estáveis e amplamente difundidas no mercado, ou adotar soluções especializadas como os bancos de dados orientados a grafos. Embora estes últimos prometam desempenho superior em consultas que envolvem múltiplos saltos relacionais, sua introdução exige uma mudança drástica de paradigma na modelagem conceitual e a adoção de novas linguagens declarativas de consulta, como a linguagem Cypher, em oposição ao tradicional SQL.

Apesar de a literatura técnica indicar que os bancos de dados de grafos superam o modelo relacional em operações de travessia de redes complexas, a quantificação exata dessa vantagem em cenários reais de comércio eletrônico ainda carece de experimentações estruturadas, isoladas e devidamente contextualizadas com datasets de produção. Adicionalmente, faz-se necessário avaliar o impacto qualitativo na legibilidade e na manutenção do código gerado por ambas as abordagens, ponderando o balanço entre ganho de velocidade e complexidade de engenharia.

Diante desse cenário de incerteza arquitetural, e considerando a necessidade premente de embasamento empírico para a tomada de decisões de projeto na área de Engenharia de Software, esta pesquisa buscou responder ao seguinte problema: Qual é o ganho real de desempenho computacional e de expressividade ao migrar a lógica de um motor de recomendação — baseado em filtragem colaborativa e em análise de perfil similar — de um banco de dados relacional (PostgreSQL) para um banco de dados orientado a grafos (Neo4j), utilizando um conjunto de dados reais do e-commerce brasileiro?

1.2 Objetivos

A definição clara do escopo e das metas desta pesquisa permitiu nortear a execução dos experimentos e a posterior análise comparativa entre as tecnologias avaliadas.

1.2.1 Objetivo Geral

O objetivo geral deste trabalho foi comparar o desempenho quantitativo, mensurado por meio do tempo de resposta de consultas, e a expressividade qualitativa, avaliada pela complexidade estrutural do código, entre o banco de dados relacional PostgreSQL e o banco de dados orientado a grafos Neo4j, aplicados a um sistema de recomendação no domínio do comércio eletrônico.

1.2.2 Objetivos Específicos

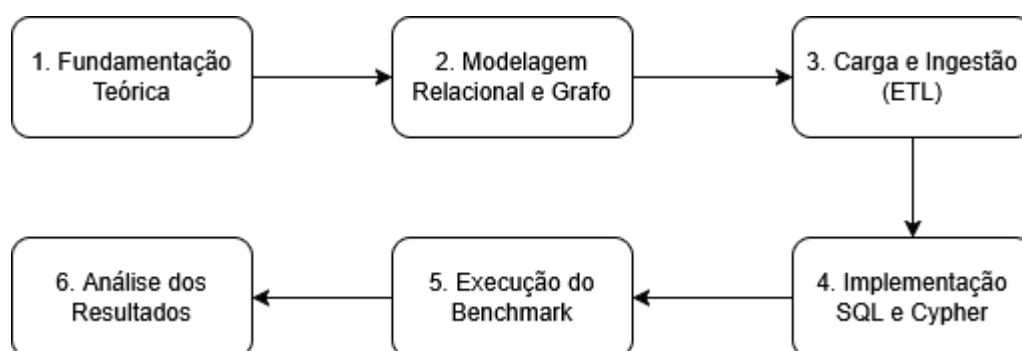
Para alcançar o objetivo geral proposto, foram estabelecidos e integralmente cumpridos os seguintes objetivos específicos ao longo do desenvolvimento do projeto:

- **Realizar o levantamento bibliográfico estruturado:** Investigar o estado da arte referente a sistemas de recomendação, abordando algoritmos de filtragem colaborativa e baseada em conteúdo, bem como os fundamentos teóricos de bancos de dados relacionais e NoSQL orientados a grafos;
- **Modelar e estruturar as bases de dados:** Adaptar e mapear o conjunto de dados público e real fornecido pela Olist para ambos os paradigmas, desenvolvendo o Diagrama Entidade-Relacionamento (DER) para o PostgreSQL e o modelo de Grafo de Propriedades Rotuladas para o Neo4j, garantindo estrita paridade informacional e volumétrica;
- **Desenvolver os componentes de carga e ingestão (ETL):** Implementar rotinas automatizadas em linguagem Python para o tratamento, conversão e inserção eficiente dos dados históricos nas duas ferramentas configuradas, validando a integridade dos registros populados;
- **Implementar as consultas de recomendação e algoritmos de rede (Queries):** Codificar cinco cenários analíticos distintos em sintaxe SQL e Cypher, cobrindo: filtragem colaborativa simples (Q1), detecção de

coocorrência (Q2), recomendação híbrida por perfil (Q3), teste de estresse escalonado por explosão combinatória (Q4) e descoberta algorítmica do caminho mais curto (Q5);

- **Executar o benchmark dinâmico sob isolamento de hardware:** Desenvolver um motor de testes em Python com proteção contra falhas (timeout de 120s), que realiza a amostragem dinâmica de chaves a cada iteração para anular o viés de cache relacional. Os tempos de resposta foram coletados em baterias de até 50 repetições controladas após a execução de ciclos de aquecimento (warm-up).
- **Analisar quantitativa e qualitativamente os resultados:** Tabular as métricas de tempo de resposta, calcular indicadores estatísticos de tendência central e dispersão (média e desvio-padrão) e avaliar a expressividade das linguagens por meio da análise de linhas de código (LOC) e complexidade sintática, gerando subsídios consolidados para a conclusão do estudo.

Figura 1: Fluxograma metodológico dos objetivos específicos



Fonte: Autoria própria (2026)

1.3 Justificativa

A escolha da tecnologia de armazenamento e gerenciamento de dados constitui uma das decisões mais críticas e determinantes na engenharia de sistemas de *software* modernos. Essa definição influencia diretamente não apenas o custo de infraestrutura computacional e de licenciamento, mas também a escalabilidade do produto e a experiência final oferecida ao usuário. No contexto altamente competitivo do comércio eletrônico, a latência na renderização de páginas e na exibição de ofertas personalizadas correlaciona-se diretamente com as taxas de abandono de carrinho e

a perda de receita financeira. Portanto, otimizar o tempo de resposta dos motores de recomendação é uma prioridade técnica e de negócio.

Sob a ótica da Engenharia de Software, esta pesquisa justificou-se pela necessidade de fornecer dados empíricos robustos e análises comparativas isentas sobre o comportamento de SGBDs de paradigmas distintos operando sobre a mesma massa de dados. Enquanto a maior parte da literatura foca em discussões teóricas ou em testes com dados sintéticos e simplificados, este estudo utilizou um *dataset* de produção real do mercado brasileiro (Olist), contendo mais de cem mil pedidos e uma complexa teia de relacionamentos geográficos e comerciais. Isso garantiu que os resultados refletissem os desafios práticos enfrentados por equipes de desenvolvimento no ambiente corporativo.

Além do aspecto puramente focado em desempenho de hardware (tempo de processamento e latência de I/O), o trabalho justificou-se pelo seu caráter qualitativo ao analisar a expressividade das linguagens de consulta. Medir a complexidade de escrita entre o SQL e o Cypher é fundamental, visto que a manutenibilidade do código-fonte e a velocidade de desenvolvimento são fatores que impactam diretamente o custo total de propriedade (*Total Cost of Ownership - TCO*) do *software*. Assim, as evidências levantadas servem como um guia de apoio à decisão para arquitetos de sistemas que planejam ou avaliam a migração de arquiteturas de dados tradicionais para modelos baseados em grafos.

1.4 Organização do Trabalho

Este Trabalho de Conclusão de Curso está estruturado de forma lógica e sequencial em seis capítulos, descritos detalhadamente a seguir:

- **Capítulo 1 - Introdução:** Apresenta as considerações iniciais que motivaram a pesquisa, estabelecendo a contextualização do problema no cenário do e-commerce, a definição formal da pergunta de pesquisa, os objetivos gerais e específicos delimitados e a justificativa técnica do estudo;
- **Capítulo 2 - Fundamentação Teórica:** Dedicar-se à revisão da literatura e à pavimentação conceitual do trabalho, detalhando o funcionamento dos bancos de dados relacionais e orientados a grafos, a teoria por trás dos sistemas de recomendação e as características específicas das ferramentas PostgreSQL e Neo4j;

- **Capítulo 3 - Metodologia:** Descreve com rigor científico o método experimental adotado, especificando detalhadamente as configurações do ambiente de hardware (Acer Nitro V15) e software utilizados, a composição do dataset da Olist e o protocolo sistemático desenhado para a coleta e o isolamento dos dados de benchmark;
- **Capítulo 4 - Modelagem e Implementação do Sistema:** Expõe detalhadamente o processo de engenharia reversa e transformação dos dados, apresentando o Diagrama Entidade-Relacionamento do modelo relacional e a topologia de nós e relacionamentos do modelo de grafos, além de detalhar as rotinas de ETL e a construção das consultas analíticas;
- **Capítulo 5 - Análise dos Resultados e Discussão:** Centraliza os achados práticos da pesquisa, confrontando os tempos de execução obtidos por meio de análise estatística descritiva e discutindo as razões arquiteturais que levaram às diferenças de performance e expressividade entre o SQL e o Cypher;
- **Capítulo 6 - Considerações Finais:** Sintetiza as conclusões obtidas face aos objetivos propostos, mapeia as limitações identificadas no ambiente controlado de testes e sugere trabalhos futuros para a continuidade e expansão da pesquisa na área de engenharia de dados.

2 FUNDAMENTAÇÃO TEÓRICA

A fundamentação teórica deste trabalho tem como objetivo estabelecer os conceitos necessários para compreender a comparação entre bancos de dados relacionais e bancos de dados orientados a grafos em consultas de recomendação para e-commerce. Para isso, são discutidos os fundamentos de bancos de dados, o modelo relacional, o movimento NoSQL, os bancos de dados orientados a grafos, os sistemas de recomendação e as ferramentas empregadas na implementação dos protótipos.

A organização desta seção segue a lógica do experimento desenvolvido. Inicialmente, apresenta-se o papel dos SGBDs e do modelo relacional, com ênfase em normalização, junções e busca por índice B-Tree, pois essa foi a estratégia adotada no Protótipo A. Em seguida, são discutidos os bancos NoSQL e, especificamente, os bancos orientados a grafos, que fundamentam o Protótipo B. Por fim, são abordados os sistemas de recomendação e suas principais abordagens de filtragem, conectando a teoria às consultas Q1, Q2, Q3, Q4 e Q5 avaliadas no benchmark.

2.1 Banco de Dados

No contexto da Engenharia de Software, um banco de dados pode ser compreendido como uma coleção organizada de dados relacionados, projetada para representar informações de um domínio de aplicação e permitir operações de armazenamento, consulta, atualização e análise. Um Sistema Gerenciador de Banco de Dados (SGBD), por sua vez, corresponde ao software responsável por administrar esses dados, oferecendo mecanismos de definição, manipulação, segurança, concorrência, integridade e recuperação após falhas (ELMASRI; NAVATHE, 2011).

A principal contribuição dos SGBDs está na separação entre a lógica da aplicação e os detalhes físicos de armazenamento. Em vez de a aplicação manipular diretamente arquivos e estruturas internas de disco, o banco oferece uma camada de abstração capaz de organizar os dados segundo um modelo específico, como o relacional, o documental, o chave-valor, o orientado a colunas ou o orientado a grafos. A escolha desse modelo influencia diretamente a forma como os dados são estruturados, consultados e otimizados pelo mecanismo interno do banco (ELMASRI; NAVATHE, 2011; SADALAGE; FOWLER, 2012).

Essa escolha torna-se especialmente relevante em sistemas que manipulam grandes volumes de dados interconectados, como plataformas de comércio eletrônico. Nesses ambientes, informações sobre clientes, pedidos, produtos, vendedores e categorias podem ser representadas de diferentes maneiras, e cada representação tende a favorecer determinados padrões de consulta. Por isso, antes de comparar empiricamente PostgreSQL e Neo4j, é necessário compreender as características estruturais dos modelos de dados que sustentam essas ferramentas.

2.2 Banco de Dados Relacional

O modelo relacional foi proposto por Edgar F. Codd em 1970 e tornou-se um dos paradigmas mais influentes da área de bancos de dados. Nesse modelo, os dados são representados logicamente por relações, implementadas na prática como tabelas compostas por linhas e colunas. Cada linha corresponde a uma tupla, e cada coluna representa um atributo da entidade modelada. A relevância do modelo relacional está na separação entre a visão lógica dos dados e os detalhes físicos de armazenamento, permitindo que o usuário consulte informações sem conhecer os caminhos internos de acesso aos registros (CODD, 1970).

Em sistemas relacionais, a consistência dos dados é mantida por meio de restrições como chaves primárias, chaves estrangeiras, domínios e regras de integridade referencial. Além disso, a normalização organiza os dados em tabelas especializadas, reduzindo redundâncias e minimizando anomalias de inserção, atualização e exclusão. No contexto deste trabalho, essa lógica aparece no Protótipo A, em que os dados da Olist foram estruturados em entidades como clientes, pedidos, produtos, vendedores e itens de pedido, preservando as associações por meio de chaves e relacionamentos tabulares.

A principal consequência dessa organização é que informações distribuídas em várias tabelas precisam ser reconstruídas durante a execução das consultas. Essa reconstrução ocorre por meio de operações de junção, ou *joins*. Em um SGBD relacional como o PostgreSQL, o otimizador de consultas avalia diferentes planos de execução, podendo utilizar estratégias como *nested loop*, *hash join* ou *merge join*, conforme a estrutura da consulta, as estatísticas disponíveis e os índices existentes.

No experimento desenvolvido, é fundamental destacar que o PostgreSQL não foi avaliado em uma configuração ingênua. Após a carga dos dados, foram criados índices estruturais do tipo B-Tree sobre colunas mapeadas como chaves estrangeiras

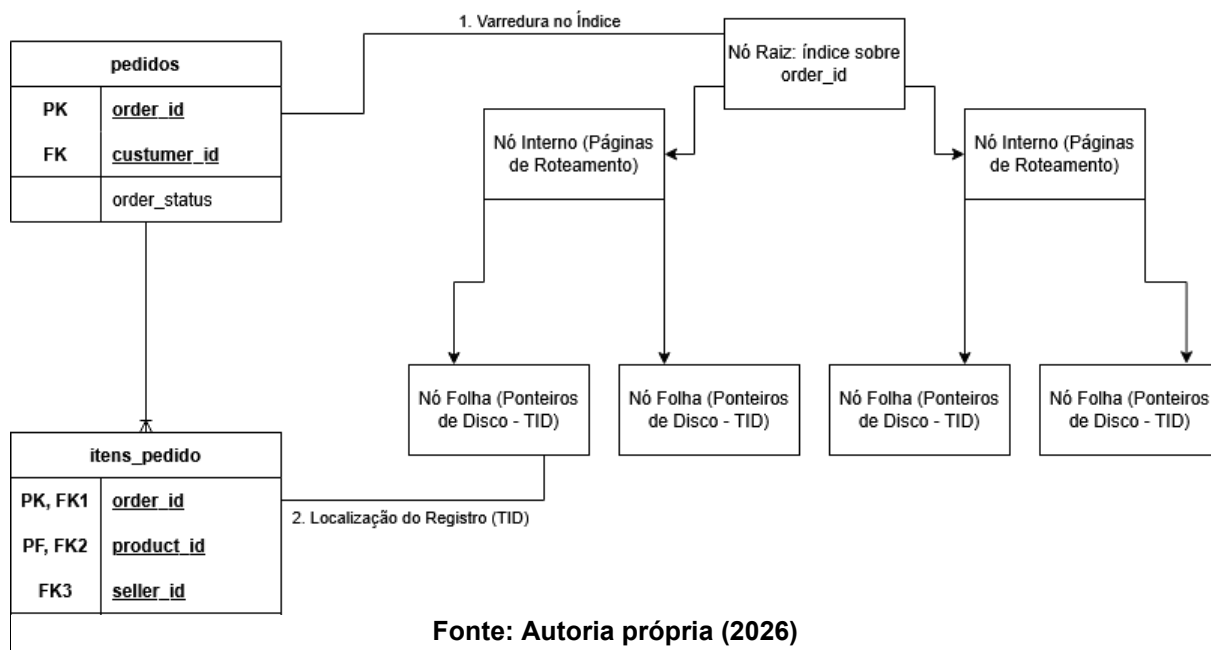
e sobre atributos utilizados em filtros lógicos, como `customer_city` e `product_category_name`. Dessa forma, as consultas relacionais foram executadas com apoio de busca indexada, permitindo que o banco localizasse registros candidatos por meio de estruturas B-Tree antes de realizar as combinações necessárias entre tabelas. Essa decisão é essencial para garantir uma comparação justa entre os paradigmas.

Assim, a limitação analisada neste trabalho não decorre da ausência de índices, mas do custo acumulado de reconstruir relações profundas por meio de junções sucessivas. Em consultas rasas e seletivas, a combinação entre otimizador relacional e índices B-Tree tende a ser altamente eficiente, o que explica o bom desempenho do PostgreSQL em cenários de baixa profundidade. No entanto, quando a consulta exige múltiplos níveis de expansão entre clientes, pedidos e produtos, o banco precisa realizar buscas sucessivas em índices e combinar conjuntos intermediários cada vez maiores.

Essa característica torna-se especialmente relevante em consultas de recomendação que simulam travessias de rede, como o teste de estresse escalonado do trabalho. À medida que a profundidade aumenta, a consulta relacional passa a envolver maior número de junções e maior complexidade de planejamento. Em cenários com muitas relações, o PostgreSQL pode recorrer ao Otimizador Genético de Consultas (GEQO), mecanismo utilizado para lidar com a explosão do espaço de planos possíveis. Portanto, o comportamento observado no benchmark deve ser interpretado como resultado da interação entre modelagem normalizada, busca por índice B-Tree, volume de dados, profundidade da consulta e complexidade do plano de execução (POSTGRESQL GLOBAL DEVELOPMENT GROUP, 2026c).

Para representar esse comportamento, a Figura 2 apresenta o mecanismo conceitual de junção e busca por índice B-Tree no modelo relacional. A figura está alinhada à lógica do Protótipo A, pois evidencia que a associação entre tabelas, como pedidos e `itens_pedido`, é resolvida por meio de busca indexada sobre identificadores e posterior localização dos registros correspondentes, diferentemente do modelo de grafos, no qual a navegação ocorre por relacionamentos explicitamente armazenados (POSTGRESQL GLOBAL DEVELOPMENT GROUP, 2026a; POSTGRESQL GLOBAL DEVELOPMENT GROUP, 2026b).

Figura 2 - Mecanismo de Junção (Join) e Busca por Índice B-Tree no Modelo Relacional



2.3 Banco de Dados NoSQL

O termo NoSQL, frequentemente interpretado como *Not Only SQL*, designa um conjunto de tecnologias de banco de dados desenvolvidas para atender demandas que nem sempre se ajustam de forma ideal ao modelo relacional tradicional. Essas demandas incluem flexibilidade de esquema, escalabilidade horizontal, armazenamento de dados semiestruturados e otimização para padrões específicos de consulta. Portanto, NoSQL não corresponde a uma única tecnologia, mas a uma família de modelos de dados com características distintas (SADALAGE; FOWLER, 2012).

Entre as principais categorias de bancos NoSQL estão os bancos chave-valor, orientados a documentos, orientados a colunas largas e orientados a grafos. Bancos chave-valor são otimizados para recuperação simples por chave; bancos documentais armazenam estruturas semiestruturadas, como JSON; bancos orientados a colunas largas favorecem determinados tipos de processamento distribuído; e bancos orientados a grafos priorizam a representação explícita de entidades e relacionamentos. Essa diversidade mostra que a adoção de NoSQL deve ser guiada pelo padrão de acesso aos dados, e não por uma oposição simplista ao modelo relacional.

A discussão sobre NoSQL também se relaciona a arquiteturas distribuídas e aos compromissos entre consistência, disponibilidade e tolerância a partição, frequentemente associados ao Teorema CAP. Contudo, para o escopo deste trabalho, o ponto mais relevante não é a distribuição dos dados em cluster, mas a adequação do modelo de dados ao tipo de consulta analisado. Como o experimento foi executado localmente e comparou consultas de recomendação sobre a mesma base, a contribuição do NoSQL neste TCC está concentrada na categoria de bancos orientados a grafos.

Nesse sentido, o NoSQL funciona como ponte conceitual entre o modelo relacional e o modelo de grafos. A comparação proposta não busca afirmar que bancos NoSQL substituem SGBDs relacionais em todos os contextos. Ao contrário, o trabalho parte da ideia de que diferentes tecnologias podem coexistir em uma arquitetura de software, cada uma aplicada ao tipo de problema para o qual apresenta maior aderência. Em um e-commerce, operações transacionais de pedido e pagamento podem continuar adequadas ao modelo relacional, enquanto consultas de recomendação baseadas em múltiplas conexões podem ser avaliadas em estruturas de grafo.

2.4 Banco de Dados Orientado a Grafos

Bancos de dados orientados a grafos representam informações por meio de nós e relacionamentos. Os nós correspondem às entidades do domínio, enquanto os relacionamentos representam as conexões entre essas entidades. Essa forma de modelagem é especialmente adequada a problemas nos quais a informação relevante não está apenas nos registros isolados, mas no modo como eles se conectam. Sistemas de recomendação, redes sociais, detecção de fraude, análise de rotas e descoberta de caminhos são exemplos de domínios frequentemente associados a grafos (ROBINSON; WEBBER; EIFREM, 2015).

O Neo4j, utilizado neste trabalho como representante do paradigma de grafos, adota o modelo de Grafo de Propriedades Rotuladas, ou *Labeled Property Graph* (LPG). Nesse modelo, nós podem receber rótulos que indicam seu tipo, como `:Cliente`, `:Pedido`, `:Produto` e `:Vendedor`; relacionamentos possuem direção e tipo, como `[:FEZ_PEDIDO]` e `[:CONTEM]`; e tanto nós quanto relacionamentos podem armazenar propriedades em formato chave-valor. Essa estrutura permite que

características das entidades e das conexões sejam representadas diretamente no grafo.

No Protótipo B, a modelagem respeitou essa lógica. As entidades relacionais fortes foram convertidas em nós, enquanto a entidade associativa `itens_pedido` foi incorporada ao relacionamento `[:CONTEM]`, que conecta pedidos e produtos. Essa decisão é coerente com o domínio do e-commerce, pois o item de pedido não atua apenas como registro isolado, mas como uma associação transacional entre um pedido e um produto. Atributos como preço e quantidade podem ser preservados como propriedades do relacionamento, mantendo a paridade informacional com o modelo relacional.

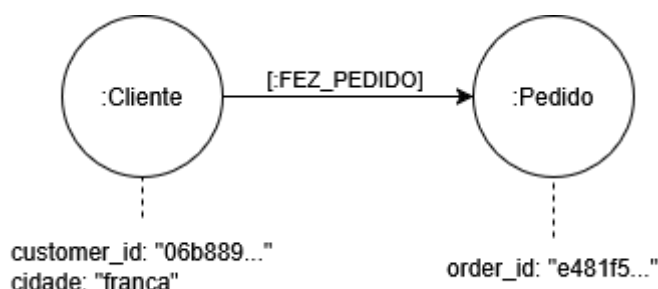
A diferença central em relação ao modelo relacional está no modo de acesso aos relacionamentos. No PostgreSQL, as associações entre tabelas são reconstruídas durante a consulta por meio de junções apoiadas por índices B-Tree. No Neo4j, os índices podem ser utilizados para localizar nós iniciais, mas a navegação subsequente ocorre por meio dos relacionamentos armazenados no grafo. Essa característica é conhecida como adjacência livre de índice (*index-free adjacency*) e favorece consultas de travessia, nas quais o objetivo é percorrer caminhos entre entidades conectadas.

Essa vantagem, contudo, deve ser formulada com precisão. O uso de grafos não torna toda consulta automaticamente mais rápida. O desempenho depende da profundidade da travessia, da seletividade dos filtros, da quantidade de relacionamentos percorridos e da presença de nós altamente conectados. Em e-commerce, produtos muito populares ou categorias muito amplas podem formar estruturas semelhantes a supernós, aumentando o volume de adjacências analisadas. Portanto, a comparação correta não é entre um banco “rápido” e outro “lento”, mas entre duas arquiteturas que respondem de forma diferente a padrões distintos de consulta.

No contexto deste trabalho, os grafos tornam-se especialmente relevantes nas consultas de maior profundidade e na descoberta de caminhos. Enquanto consultas rasas podem ser resolvidas de maneira eficiente por junções indexadas no PostgreSQL, consultas que exigem expansão progressiva de vizinhança ou cálculo de menor caminho tendem a se alinhar melhor ao modelo de grafos. Essa relação justifica a presença de consultas como Q4, voltada ao teste de estresse por múltiplos saltos, e Q5, voltada à descoberta de menor caminho.

A Figura 3 apresenta a estrutura conceitual de um grafo de propriedades rotuladas utilizada como base para o Protótipo B. A figura está coerente com a modelagem do trabalho, pois representa nós rotulados e relacionamentos direcionados, evidenciando que a associação entre entidades é armazenada explicitamente no grafo, e não reconstruída posteriormente por junções tabulares.

Figura 3 - Estrutura Conceitual de um Grafo de Propriedades Rotuladas (LPG)



Fonte: Autoria própria (2026)

2.5 Sistemas de Recomendação

Sistemas de Recomendação (SR) são mecanismos computacionais utilizados para sugerir itens potencialmente relevantes a usuários, com base em dados de preferência, histórico de interação, similaridade entre perfis ou características dos próprios itens. Esses sistemas tornaram-se especialmente importantes em plataformas de comércio eletrônico, nas quais o volume de produtos disponíveis pode dificultar a busca manual e a tomada de decisão por parte do consumidor (RICCI; ROKACH; SHAPIRA, 2011; AGGARWAL, 2016).

De forma geral, um sistema de recomendação procura estimar a utilidade de um item ainda não consumido por determinado usuário. Essa estimativa pode ser construída a partir de avaliações explícitas, como notas e comentários, ou de sinais implícitos, como cliques, visualizações, adições ao carrinho e compras. No contexto deste trabalho, o histórico de compras assume papel central, pois o conjunto de dados utilizado registra pedidos reais de e-commerce, incluindo clientes, produtos, vendedores e itens adquiridos.

Uma forma clássica de representar o problema de recomendação é por meio de uma matriz de interação entre usuários e itens. Nessa matriz, as linhas representam usuários, as colunas representam itens e as células indicam alguma forma de interação, como compra, avaliação ou interesse observado. Como cada

usuário interage apenas com uma pequena parcela do catálogo total, essa matriz tende a ser esparsa, exigindo técnicas capazes de inferir relações ainda não observadas a partir dos dados disponíveis (RICCI; ROKACH; SHAPIRA, 2011).

No domínio de e-commerce, as recomendações podem assumir diferentes formas, como sugerir produtos semelhantes aos já comprados, indicar itens frequentemente adquiridos em conjunto ou apresentar produtos consumidos por clientes com perfis parecidos. Essas estratégias dependem da exploração de relações entre usuários, pedidos, produtos e categorias. Por isso, o desempenho do sistema de recomendação não depende apenas da lógica algorítmica adotada, mas também da forma como essas relações são armazenadas e consultadas.

Essa relação entre algoritmo e persistência é o ponto central deste TCC. A mesma lógica de recomendação pode ser implementada em um banco relacional, por meio de junções entre tabelas apoiadas por índices B-Tree, ou em um banco orientado a grafos, por meio de travessias entre nós e relacionamentos. Assim, o objetivo deste trabalho não é propor um novo algoritmo de recomendação, mas comparar como PostgreSQL e Neo4j executam consultas representativas desse domínio em condições controladas de benchmark.

A partir dessa perspectiva, as consultas elaboradas no experimento representam diferentes níveis de complexidade em sistemas de recomendação. As consultas Q1 e Q2 avaliam recomendações de baixa profundidade, relacionadas à filtragem colaborativa e à coocorrência de produtos. A consulta Q3 representa uma estratégia híbrida, combinando similaridade de perfil, localização geográfica e categoria de produto. As consultas Q4 e Q5, por sua vez, exploram cenários de maior profundidade topológica, nos quais a expansão de vizinhança e a descoberta de caminhos tornam mais evidente a diferença entre reconstruir relações por junções e percorrê-las diretamente em um grafo.

2.6 Filtragem Colaborativa

A filtragem colaborativa é uma das abordagens mais consolidadas em sistemas de recomendação. Seu princípio central é que usuários que apresentaram comportamentos semelhantes no passado tendem a ter interesses semelhantes no futuro. Em vez de depender exclusivamente da descrição interna dos itens, essa abordagem utiliza o histórico coletivo de interações para identificar padrões de

similaridade entre usuários ou entre produtos (RICCI; ROKACH; SHAPIRA, 2011; AGGARWAL, 2016).

Essa técnica é particularmente relevante em comércio eletrônico, pois o histórico de compras permite observar relações de afinidade entre consumidores e produtos. Por exemplo, se diversos clientes que compraram um determinado item também compraram outro produto, o sistema pode inferir uma associação entre esses itens. Da mesma forma, se dois clientes possuem históricos de compra semelhantes, produtos adquiridos por um deles podem ser recomendados ao outro. Essas inferências são construídas a partir das interações registradas, sem exigir necessariamente uma descrição semântica detalhada de cada produto.

A literatura costuma dividir a filtragem colaborativa em duas perspectivas principais. Na abordagem baseada em usuários, o sistema identifica consumidores com histórico semelhante ao do usuário alvo e recomenda itens consumidos por esses vizinhos. Na abordagem baseada em itens, o foco está na relação entre os próprios produtos, identificando itens frequentemente consumidos em conjunto ou por públicos semelhantes. Ambas dependem da exploração de relações entre usuários, itens e eventos de interação, o que torna o armazenamento dessas relações um fator relevante para o desempenho.

No experimento desenvolvido neste trabalho, essa lógica aparece principalmente nas consultas Q1 e Q2. A Q1 representa uma recomendação colaborativa simples: dado um produto alvo, busca outros produtos comprados pelos mesmos clientes que adquiriram esse item. Já a Q2 corresponde a uma análise de coocorrência, identificando produtos frequentemente adquiridos no mesmo pedido. No modelo relacional, essas consultas exigem junções entre tabelas, com apoio dos índices B-Tree criados sobre chaves e atributos relevantes. No modelo de grafos, a mesma lógica é expressa por travessias entre nós `:Produto`, `:Pedido` e `:Cliente`, conectados por relacionamentos como `[:CONTEM]` e `[:FEZ_PEDIDO]`.

Essa distinção é essencial para interpretar os resultados. Em consultas rasas e bem seletivas, o PostgreSQL pode apresentar desempenho superior justamente porque utiliza um otimizador maduro e estruturas de índice B-Tree para localizar rapidamente os registros envolvidos. Portanto, quando Q1 e Q2 envolvem poucos saltos lógicos e conjuntos intermediários controlados, o modelo relacional não deve ser tratado como inferior; pelo contrário, ele tende a ser altamente competitivo e, em determinados cenários, mais eficiente que a travessia em grafo.

Por outro lado, conforme a recomendação passa a depender de múltiplos níveis de vizinhança, o custo acumulado das junções tende a crescer. Nesse caso, a vantagem potencial dos grafos está na representação explícita das conexões, permitindo que a consulta seja formulada como um percurso entre entidades conectadas. Essa característica torna a filtragem colaborativa um ponto adequado para comparar expressividade e desempenho entre SQL e Cypher, pois a mesma regra de negócio pode ser escrita como uma sequência de junções ou como um padrão de travessia.

Apesar de sua utilidade, a filtragem colaborativa possui limitações. Uma delas é o problema do início frio (*cold start*), que ocorre quando novos usuários ou novos produtos ainda não possuem interações suficientes para gerar recomendações confiáveis. Outra limitação é a esparsidade da matriz usuário-item: quanto menor a quantidade de interações observadas, mais difícil se torna identificar vizinhos ou produtos semelhantes com alto grau de confiança (RICCI; ROKACH; SHAPIRA, 2011; AGGARWAL, 2016).

Para sintetizar a lógica dessa abordagem, a Figura 4 apresenta a matriz de interação usuário-item e o mecanismo de dedução utilizado pela filtragem colaborativa. A figura está alinhada ao escopo do trabalho porque representa a inferência de recomendações a partir de padrões de consumo compartilhados, que é a base conceitual das consultas colaborativas avaliadas no benchmark.

Figura 4 - Matriz de Interação e Mecanismo de Dedução por Filtragem Colaborativa

	Produto 1	Produto 2	Produto 3	Produto 4
Usuário A	X	X		
Usuário B	X		X	X
Usuário C			X	
Usuário Alvo	X	X		

Recomendação Inferida

Fonte: Autoria própria (2026)

2.7 Filtragem Baseada em Conteúdo

A filtragem baseada em conteúdo adota uma lógica diferente da filtragem colaborativa. Em vez de utilizar principalmente o comportamento coletivo dos usuários, essa abordagem recomenda itens semelhantes àqueles pelos quais o usuário já demonstrou interesse. Para isso, considera características dos próprios produtos, como categoria, marca, descrição, preço, especificações técnicas ou outros metadados disponíveis no catálogo (RICCI; ROKACH; SHAPIRA, 2011; AGGARWAL, 2016).

Em ambientes de comércio eletrônico, a filtragem baseada em conteúdo é relevante porque os produtos costumam possuir atributos descritivos que podem ser usados para estimar similaridade. Se um usuário compra com frequência itens de determinada categoria, o sistema pode priorizar produtos pertencentes a categorias próximas ou equivalentes. Diferentemente da filtragem colaborativa, essa abordagem não depende exclusivamente de outros usuários com comportamento semelhante, pois utiliza informações internas dos itens para orientar a recomendação.

Uma vantagem importante da filtragem baseada em conteúdo é sua utilidade em situações nas quais há poucos dados coletivos disponíveis. Produtos novos, por exemplo, podem ainda não ter sido adquiridos por muitos clientes, mas já possuem categoria, descrição e outros atributos cadastrados. Esses metadados permitem que o sistema os recomende com base na proximidade em relação ao perfil de consumo do usuário. Essa característica ajuda a reduzir parte do problema de início frio associado a novos itens (AGGARWAL, 2016).

Entretanto, a abordagem baseada em conteúdo também apresenta limitações. Como tende a recomendar itens semelhantes aos que o usuário já consumiu, ela pode reduzir a diversidade das recomendações e limitar a descoberta de produtos fora do padrão histórico do consumidor. Por esse motivo, sistemas reais frequentemente combinam diferentes estratégias, formando recomendações híbridas que utilizam tanto padrões coletivos de interação quanto características internas dos itens.

No contexto deste trabalho, essa combinação aparece na consulta Q3. A Q3 não representa uma filtragem puramente colaborativa nem uma filtragem puramente baseada em conteúdo. Ela busca clientes relacionados por critérios de perfil e geografia e, ao mesmo tempo, restringe os produtos recomendados às categorias

pelas quais o cliente alvo já demonstrou interesse. Assim, a consulta combina comportamento de usuários semelhantes com metadados dos produtos, aproximando-se de uma estratégia híbrida de recomendação.

Essa formulação é importante para o benchmark porque aumenta a complexidade da consulta sem abandonar um cenário plausível de e-commerce. No PostgreSQL, a Q3 depende de junções entre tabelas e filtros sobre atributos como cidade do cliente e categoria do produto, beneficiando-se dos índices B-Tree criados sobre esses campos. No Neo4j, a consulta combina travessias entre nós com restrições sobre propriedades, o que pode gerar custos adicionais quando há necessidade de percorrer ou agregar conjuntos maiores antes de aplicar todos os filtros. Dessa forma, a Q3 permite observar um caso intermediário entre consultas rasas de coocorrência e consultas profundas de expansão de rede.

Portanto, a filtragem baseada em conteúdo sustenta a parte do experimento que incorpora atributos dos produtos à recomendação. Sua função no TCC não é deslocar o foco para algoritmos textuais, vetoriais ou de aprendizado de máquina, mas justificar o uso de metadados, especialmente a categoria do produto, como critério complementar à filtragem colaborativa.

2.8 Ferramentas Utilizadas

A execução empírica deste trabalho exigiu a seleção de ferramentas capazes de representar adequadamente os dois paradigmas comparados e permitir a automação do processo de carga, consulta e medição de desempenho. A escolha das tecnologias considerou maturidade, adoção prática, disponibilidade para uso acadêmico, integração com Python e aderência ao tipo de modelagem necessário para o experimento.

Foram utilizadas três bases tecnológicas principais. O PostgreSQL representou o modelo relacional, permitindo a criação de tabelas normalizadas, chaves, restrições e índices B-Tree. O Neo4j representou o modelo de grafos, permitindo a construção de um Grafo de Propriedades Rotuladas com nós, relacionamentos e propriedades. A linguagem Python foi empregada como camada de orquestração, responsável por executar rotinas de ETL, conectar-se aos bancos, disparar consultas e registrar os tempos de resposta.

A função desta seção não é repetir toda a fundamentação sobre modelos relacionais e grafos, mas apresentar como essas ferramentas foram aplicadas no

experimento. As características conceituais dos paradigmas já foram discutidas nas subseções anteriores; neste ponto, o foco recai sobre o papel de cada tecnologia na implementação dos protótipos e na coleta das métricas do benchmark.

2.8.1 PostgreSQL

O PostgreSQL é um Sistema Gerenciador de Banco de Dados Objeto-Relacional de código aberto, amplamente utilizado em aplicações que exigem integridade, estabilidade e suporte avançado à linguagem SQL. Neste trabalho, ele foi selecionado como representante do modelo relacional por permitir a modelagem normalizada dos dados da Olist e por oferecer mecanismos consolidados de consulta, indexação e otimização.

No Protótipo A, o PostgreSQL armazenou os dados em tabelas relacionadas, preservando entidades como clientes, pedidos, produtos, vendedores e itens de pedido. Essa estrutura permitiu representar o dataset no formato relacional, com associações entre tabelas por meio de chaves. Após a carga dos dados, foram criados índices B-Tree sobre chaves estrangeiras e atributos de filtro, garantindo que as consultas fossem executadas com apoio de busca indexada e não por simples varreduras completas de tabela. Essa decisão foi essencial para que o PostgreSQL atuasse como uma base de comparação justa e tecnicamente otimizada.

Durante o benchmark, o PostgreSQL foi responsável por executar as versões SQL das consultas Q1 a Q5. Nas consultas de baixa profundidade, como Q1 e Q2, a combinação entre junções seletivas e índices B-Tree permite explorar a força do modelo relacional em cenários tabulares bem estruturados. Já nos testes de maior profundidade, como Q4 e Q5, o banco foi submetido a consultas com maior número de junções e maior complexidade de planejamento, o que permitiu avaliar os limites da abordagem relacional diante de padrões de recomendação mais conectados.

2.8.2 Neo4j e Cypher

O Neo4j é um banco de dados orientado a grafos nativo, utilizado neste trabalho como representante do paradigma de grafos. Sua escolha está associada à adoção do modelo de Grafo de Propriedades Rotuladas, no qual entidades são representadas como nós, associações são representadas como relacionamentos e atributos podem ser armazenados como propriedades. Essa estrutura é adequada ao

domínio analisado, pois sistemas de recomendação dependem de conexões entre clientes, pedidos, produtos e vendedores.

No Protótipo B, os principais elementos do dataset foram convertidos para nós rotulados, como `:Cliente`, `:Pedido`, `:Produto` e `:Vendedor`. As associações entre essas entidades foram representadas por relacionamentos, como `[:FEZ_PEDIDO]` e `[:CONTEM]`. A entidade associativa `itens_pedido`, presente no modelo relacional, foi incorporada ao relacionamento `[:CONTEM]`, preservando seus atributos como propriedades. Essa decisão manteve a paridade informacional com o PostgreSQL, mas adaptou a estrutura ao modo nativo de representação do grafo.

A linguagem Cypher foi utilizada para formular as consultas no Neo4j. Sua sintaxe permite expressar padrões de conexão de forma próxima à topologia do grafo, o que favorece a legibilidade em consultas de travessia. Por exemplo, enquanto uma consulta SQL precisa reconstruir relações por meio de múltiplos JOINS, uma consulta Cypher pode representar o percurso entre cliente, pedido e produto como um padrão explícito de nós e relacionamentos. Essa diferença é especialmente relevante nas consultas Q4 e Q5, em que a expansão de vizinhança e a descoberta de caminhos são parte central da lógica avaliada.

No entanto, a avaliação do Neo4j neste trabalho não parte da suposição de superioridade universal dos grafos. A modelagem em grafo tende a favorecer consultas com múltiplos saltos e relações explícitas, mas seu desempenho também pode ser afetado por fatores como quantidade de relacionamentos percorridos, baixa seletividade de filtros e presença de nós altamente conectados. Por isso, o benchmark foi construído com amostragem dinâmica, permitindo observar variações reais de desempenho em diferentes regiões da rede de dados.

2.8.3 Linguagem Python e Ecossistema de Dados

A linguagem Python foi utilizada como camada de automação do experimento. Sua função principal foi integrar as etapas de extração, transformação, carga e benchmark, garantindo que os dois bancos fossem alimentados a partir da mesma origem de dados e submetidos a um protocolo de medição equivalente.

Na etapa de ETL, a biblioteca `pandas` foi empregada para leitura e tratamento dos arquivos CSV da Olist. Essa etapa permitiu padronizar os dados antes da carga em cada banco, preservando a equivalência informacional entre os protótipos. Em

seguida, scripts específicos foram utilizados para inserir os dados no PostgreSQL e no Neo4j, respeitando a modelagem própria de cada paradigma.

Na etapa de benchmark, Python foi utilizado para conectar-se aos bancos por meio dos drivers adequados, executar as consultas, controlar repetições, aplicar aquecimento e registrar os tempos de resposta. O uso de uma mesma camada de orquestração contribuiu para a padronização do experimento, pois as consultas foram disparadas de forma controlada a partir do mesmo ambiente de execução.

Além disso, a implementação em Python permitiu aplicar mecanismos importantes para a validade do experimento, como a amostragem dinâmica de chaves a cada iteração e o tratamento de falhas por timeout. A amostragem dinâmica reduziu o risco de enviesamento por cache ao evitar que as consultas fossem executadas sempre sobre os mesmos clientes ou produtos. Já o controle de timeout impediu que falhas em consultas de estresse comprometessem a execução geral do benchmark. Esses procedimentos são coerentes com a metodologia descrita no TCC e reforçam a confiabilidade da comparação entre os protótipos.

3 METODOLOGIA

Este capítulo descreve minuciosamente os procedimentos metodológicos que foram adotados para atingir os objetivos propostos pela pesquisa. A metodologia estabelecida define a abordagem científica da investigação, detalha as ferramentas e tecnologias selecionadas para o desenvolvimento dos protótipos, caracteriza o conjunto de dados utilizado como objeto de estudo empírico e documenta a arquitetura rigorosa do experimento comparativo, garantindo que os testes de desempenho (*benchmark*) sejam válidos e reproduzíveis.

3.1 Abordagem de Pesquisa

Quanto à sua natureza, esta pesquisa foi classificada como aplicada, uma vez que buscou gerar conhecimentos direcionados à solução de um problema prático e latente na Engenharia de Software: a escolha arquitetural da tecnologia de banco de dados para sustentar sistemas de recomendação em cenários de alta conectividade.

Em relação aos objetivos, a investigação caracterizou-se como exploratória e descritiva. Exploratória pois analisou o *trade-off* (relação de compromisso) de desempenho entre as abordagens relacional e orientada a grafos, um campo que ainda demanda experimentações específicas com dados reais do mercado nacional. Descritiva porque documentou de forma detalhada o comportamento dinâmico e o tempo de resposta das consultas sob condições estritas de controle.

Quanto à abordagem do problema, a pesquisa adotou um método misto, combinando análises quantitativas e qualitativas. As métricas de desempenho dos bancos de dados — como o tempo de resposta em milissegundos e o desvio-padrão das execuções — foram coletadas e analisadas sob um rigoroso viés quantitativo e estatístico. Em contrapartida, aspectos de engenharia como a complexidade de implementação, o processo de modelagem e a expressividade sintática das linguagens SQL e Cypher foram avaliados qualitativamente.

O método central que guiou a execução do trabalho foi o experimento de laboratório controlado (*in vitro*). Para isso, dois protótipos isolados de *software* (Protótipo A: Relacional e Protótipo B: Grafo) foram desenvolvidos em paralelo e submetidos ao exato mesmo conjunto de testes, garantindo que as variáveis de *hardware* e de volume de dados fossem mantidas idênticas, permitindo uma comparação justa (*apples-to-apples*).

3.2 Ferramentas e Tecnologias

Para a execução técnica e estrutural da pesquisa, todas as ferramentas e linguagens selecionadas possuíam código aberto (*open-source*) ou licenciamento gratuito para uso acadêmico, assegurando a total reprodutibilidade do experimento por outros pesquisadores da área. A pilha tecnológica (*tech stack*) adotada foi composta pelos seguintes elementos:

- **Banco de Dados Relacional (Protótipo A):** Foi utilizado o PostgreSQL. A escolha deste Sistema de Gerenciamento de Banco de Dados Objeto-Relacional (SGBDOR) justificou-se pela sua maturidade de mercado, robustez algorítmica, vasta adoção pela indústria no tratamento de dados de *e-commerce* e rigorosa conformidade com o padrão internacional SQL.
- **Banco de Dados de Grafo (Protótipo B):** Foi adotado o Neo4j Desktop. Trata-se da plataforma líder global no segmento de bancos de dados de grafos nativos. Sua escolha pautou-se na implementação do modelo *Labeled Property Graph* (LPG), na capacidade de navegação livre de índice (*index-free adjacency*) e na utilização da linguagem de consulta declarativa Cypher.
- **Orquestração, ETL e Benchmark:** A linguagem de programação Python foi eleita para automatizar todas as rotinas sistêmicas. A biblioteca `pandas` foi empregada no processo de Extração, Transformação e Carga (ETL), processando os arquivos CSV brutos. Os *scripts* de *benchmark*, responsáveis por executar as consultas e cronometrar os tempos de resposta com precisão, foram desenvolvidos utilizando as bibliotecas nativas de conexão `psycopg2` (para interação direta com o PostgreSQL) e `neo4j` (para comunicação via protocolo Bolt com o motor de grafos).
- **Ambiente de Desenvolvimento e Modelagem:** A escrita dos códigos ocorreu no editor Visual Studio Code (VS Code). A gestão do acervo bibliográfico foi realizada através do *software* Zotero. Para a engenharia reversa conceitual e a criação de todos os diagramas arquiteturais e de modelagem (como o DER e o modelo LPG), empregou-se a ferramenta *online* e colaborativa `diagrams.net` (antigo `draw.io`).

3.2.1 Ambiente de Execução

O isolamento de *hardware* é uma premissa crítica na execução de testes de desempenho computacional. Para garantir que nenhum gargalo externo de rede interferisse nas métricas, o experimento foi conduzido localmente em uma única máquina, forçando ambos os SGBDs a competirem e utilizarem os mesmos recursos físicos sob as mesmas condições térmicas e de energia. A estação de trabalho configurada para atuar como servidor de banco de dados e executor do *benchmark* apresentou as seguintes especificações de *hardware* e *software*:

- **Equipamento:** Notebook modelo Acer Nitro V15 (ANV15-52-790M).
- **Processador (CPU):** Intel Core i7-13620H de 13ª geração, arquitetura de 10 núcleos e 16 *threads*, com frequência turbo máxima de até 4.9 GHz.
- **Memória RAM:** 16 GB de capacidade com tecnologia DDR5, garantindo alta largura de banda para o carregamento do *cache* de banco de dados.
- **Armazenamento Secundário:** Unidade de Estado Sólido (SSD) de 512 GB com interface NVMe, minimizando a latência nas operações de leitura e gravação em disco (*I/O*).
- **Aceleração Gráfica (GPU):** NVIDIA GeForce RTX 4060 com 8 GB GDDR6 (embora o processamento do *benchmark* tenha se concentrado estritamente na CPU e no Disco).
- **Sistema Operacional:** Microsoft Windows 11 Pro (arquitetura de 64 *bits*).

Esta configuração computacional de alto desempenho assegurou um ambiente de execução consistente, provendo poder de processamento paralelo e memória volátil suficientes para suportar consultas complexas sobre grandes volumetrias de dados, sem sofrer esgotamento prematuro de recursos (*out-of-memory*) ou estrangulamento severo (*throttling*) de CPU durante as iterações do teste.

3.3 Objeto de Estudo

A validade de um experimento de desempenho em bancos de dados depende intrinsecamente da qualidade, da volumetria e da complexidade estrutural dos dados submetidos ao estresse computacional. Para garantir que os testes desta pesquisa refletissem os desafios práticos enfrentados pela indústria de *software*, descartou-se o uso de *datasets* sintéticos (gerados artificialmente). O objeto de estudo empírico

selecionado foi o conjunto de dados público *Brazilian E-Commerce Public Dataset by Olist*¹.

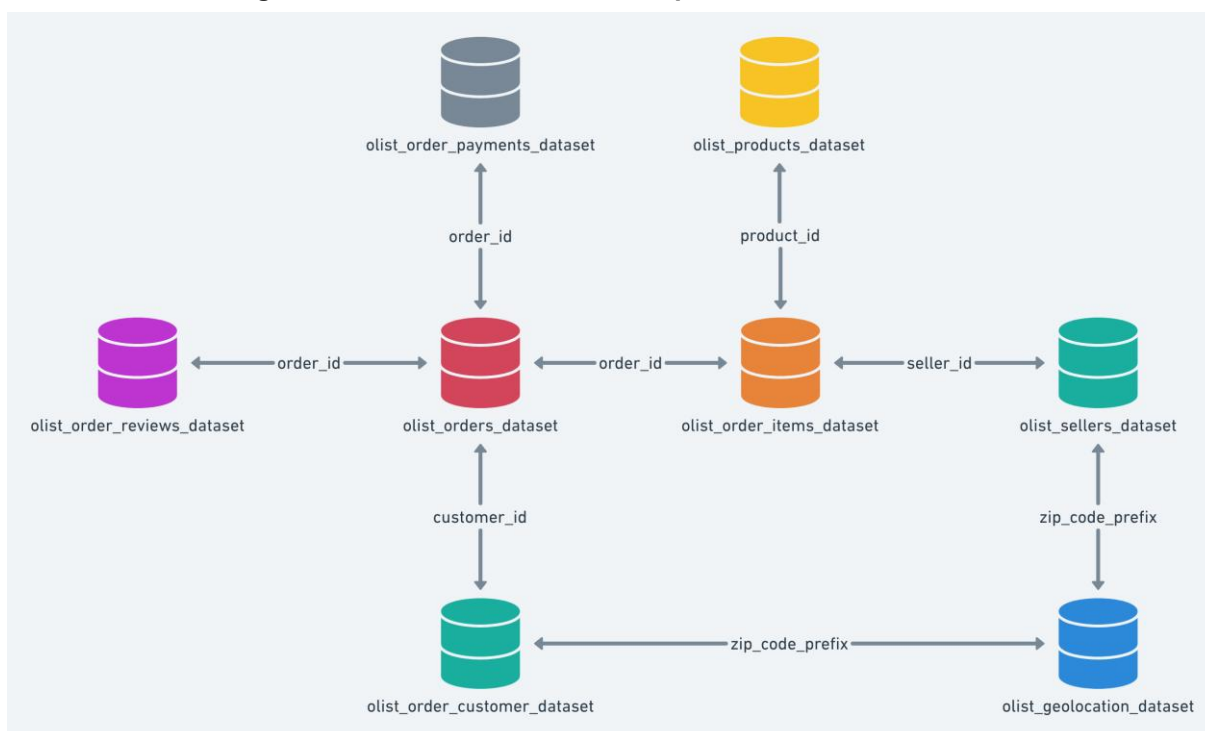
Disponibilizado abertamente na plataforma de ciência de dados *Kaggle*, este *dataset* engloba um acervo massivo de informações anonimizadas referentes a mais de 100.000 pedidos reais realizados na plataforma da Olist entre os anos de 2016 e 2018. A escolha deste conjunto de dados específico justificou-se por três pilares metodológicos centrais:

1. **Reprodutibilidade Científica:** Por se tratar de um repositório público e amplamente documentado, o uso desse *dataset* garante que qualquer engenheiro ou pesquisador possa replicar este experimento, baixar a mesma carga de dados e validar os resultados de *benchmark* obtidos.
2. **Volumetria Relevante:** A marca de cem mil pedidos interligados a milhares de clientes, produtos e vendedores gera um volume de registros robusto o suficiente para avaliar o comportamento dos mecanismos de cache, indexação B-Tree e planejamento de consultas em cenários de recomendação com múltiplas entidades relacionadas, permitindo uma medição clara de latência.
3. **Complexidade Topológica:** O esquema de dados da Olist é naturalmente concebido sob o paradigma relacional. O arquivo original é fragmentado em nove arquivos em formato *Comma-Separated Values* (CSV), altamente normalizados e interconectados por chaves estrangeiras. Essa fragmentação tornou o *dataset* um cenário de teste ideal, pois exigiu um esforço real de engenharia reversa para a adaptação ao modelo de grafos e forçou o PostgreSQL a lidar com as junções de múltiplas tabelas.

Para mapear a origem da informação antes da etapa de transformação (ETL), a Figura 5 ilustra a rede de arquivos que compõem o escopo do *dataset* e como eles se inter-relacionam através de seus identificadores.

¹ Disponível em: <https://www.kaggle.com/datasets/olistbr/brazilian-ecommerce>

Figura 5 - Estrutura Relacional Simplificada do Dataset Olist



Fonte: Kaggle (s.d.)

3.4 Arquitetura do Experimento

Para assegurar a imparcialidade técnica e o isolamento de variáveis concorrentes, a metodologia de comparação baseou-se em um fluxo de trabalho experimental rigoroso, dividido em duas trilhas arquiteturais paralelas (Protótipo A e Protótipo B). Ambas as trilhas partiram da exata mesma fonte de dados brutos e foram submetidas às mesmas restrições sistêmicas, divergindo unicamente na tecnologia de persistência empregada. O fluxo de execução do experimento foi segmentado nas quatro macroetapas descritas a seguir:

1. **Extração e Transformação (ETL):** Utilizando a linguagem Python e a biblioteca pandas, os arquivos CSV da Olist foram lidos, limpos (com a remoção de colunas sobressalentes ao domínio do *benchmark*) e padronizados na memória temporária da aplicação.
2. **Carga (Load):** O motor de ETL foi desmembrado em dois *scripts* isolados (*carga_postgres.py* e *carga_neo4j.py*). O primeiro conectou-se ao PostgreSQL, criando as tabelas normalizadas e inserindo os dados com garantia de integridade transacional. O segundo conectou-se ao Neo4j, convertendo as

linhas dos arquivos em Nós iterativos e processando as chaves estrangeiras para criar os Relacionamentos físicos ([:FEZ_PEDIDO], [:CONTEM]), mapeando o Grafo de Propriedades Rotuladas.

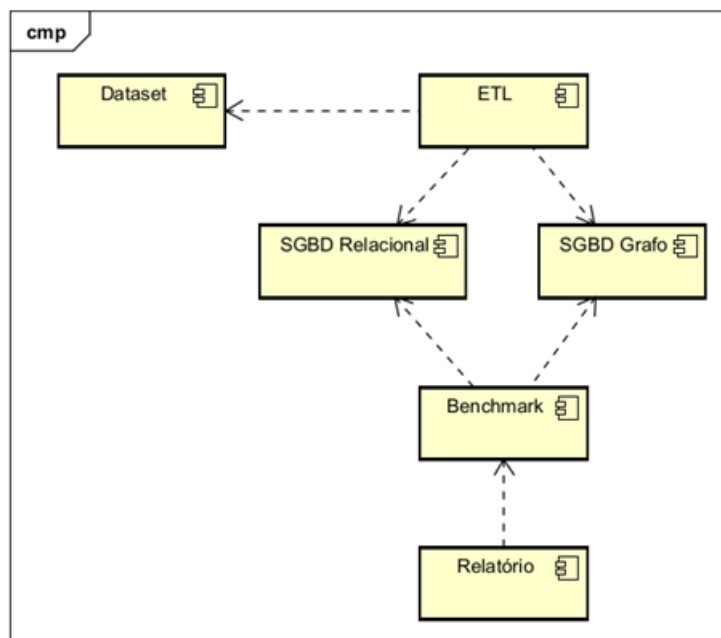
3. **Execução do Benchmark Dinâmico e Degradação:** A aplicação orquestradora (benchmark.py) assumiu o controle do experimento. Diferentemente de testes estáticos, o motor implementou uma lógica de amostragem dinâmica, sorteando aleatoriamente chaves de clientes e produtos a partir de uma base de entidades altamente conectadas no início de cada iteração. Essa abordagem impediu os SGBDs de viciarem o plano de execução na memória, simulando a concorrência orgânica de um ambiente em produção. Para testes de estresse profundo, aplicou-se um mecanismo de segurança de *Statement Timeout* de 120 segundos e tratamento de exceções, permitindo que o experimento operasse de forma resiliente mesmo após o colapso combinatório de consultas relacionais. A captura do tempo foi feita na camada de aplicação utilizando contadores de alta precisão do Python..
4. **Coleta e Análise de Resultados:** Para cada ciclo de execução, a latência cronometrada foi salva em arquivos de relatório CSV consolidados, permitindo a tabulação dos dados, a extração de médias e a posterior renderização gráfica para as análises quantitativas.

O rigor estatístico do experimento exigiu o controle minucioso do comportamento interno dos processadores e dos mecanismos de *cache* dos bancos de dados. Se uma consulta fosse executada apenas uma vez, o tempo capturado refletiria uma penalidade severa de Entrada/Saída (I/O de leitura em disco a frio) ou o tempo gasto pelo SGBD para otimizar o plano de execução, o que invalidaria a medição do tempo real de processamento de rede e CPU.

Visando contornar distorções geradas por I/O frio de disco, aplicou-se o método de Aquecimento (Warm-up). Cada consulta do benchmark (das analíticas Q1 à complexa Q5) foi programada para rodar em loop. As primeiras 5 execuções de cada teste foram processadas, mas seus tempos foram silenciosamente descartados para estabilizar o estado de *Warm Cache*. Imediatamente após o aquecimento, as execuções sequenciais subsequentes (parametrizadas em até 50 repetições para análises rasas e 10 repetições de teto para o estresse profundo) foram rigorosamente cronometradas, compondo a amostra populacional para o cálculo das métricas finais.

Para ilustrar de forma sistemática a orquestração desde os dados brutos até a emissão do relatório final, a Figura 6 apresenta a arquitetura geral em diagrama de blocos de *software*.

Figura 6 Arquitetura Geral e Fluxo de Execução do Experimento



Fonte: Autoria Própria (2025)

4 PROPOSTA DO SISTEMA

Este capítulo detalha as decisões de engenharia de *software* e a arquitetura adotada na construção dos dois protótipos de banco de dados. A modelagem foi desenhada para garantir a paridade informacional entre os paradigmas relacional e de grafos, enquanto o motor de automação do *benchmark* foi construído para expor os limites físicos de processamento, capturando discrepâncias reais de execução.

4.1 Modelagem de Dados

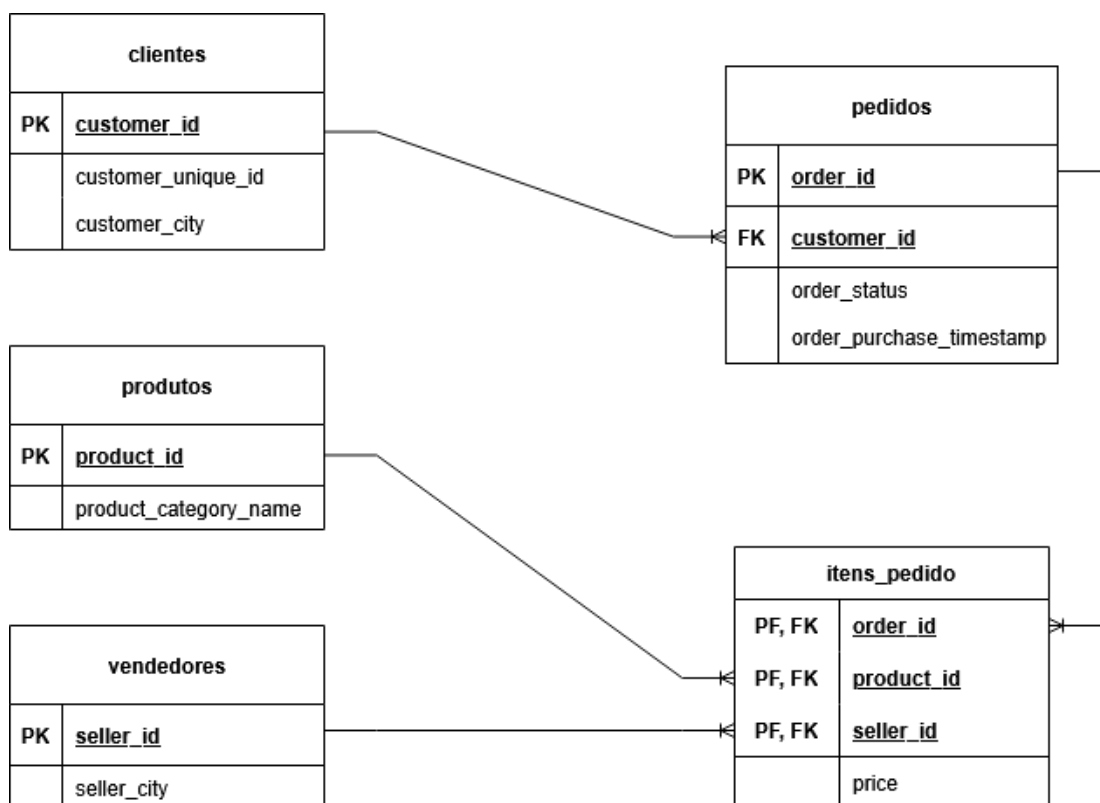
A validade científica do experimento comparativo exigiu que os dados brutos da Olist fossem mapeados para os formatos nativos de cada paradigma, respeitando as restrições e as otimizações esperadas na área de dados para sistemas em produção.

4.1.1 Protótipo A: Modelo Relacional (PostgreSQL)

A arquitetura do Protótipo A seguiu a normalização relacional até a Terceira Forma Normal (3FN). O esquema consolidou-se em quatro entidades fortes (`clientes`, `produtos`, `vendedores`, `pedidos`) interligadas pela entidade associativa `itens_pedido`, resolvendo a cardinalidade muitos-para-muitos.

Para nivelar a comparação e evitar degradação por varreduras sequenciais (*Sequential Scans*), aplicou-se a otimização física pós-carga. Foram criados índices estruturais do tipo B-Tree sobre todas as colunas mapeadas como Chaves Estrangeiras (FK) e sobre os atributos de filtro lógico (`customer_city` e `product_category_name`), garantindo que o PostgreSQL operasse em sua capacidade máxima de recuperação em memória.

Figura 7 - Diagrama Entidade-Relacionamento do Protótipo A (Relacional)



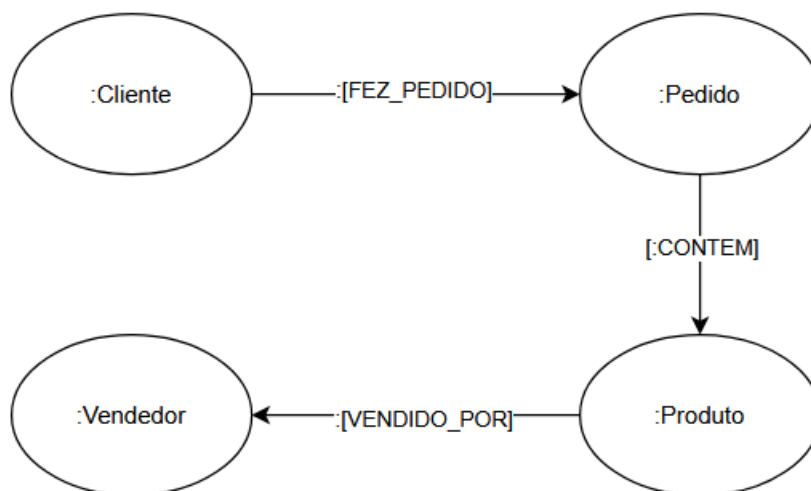
Fonte: Autoria própria (2025)

4.1.2 Protótipo B: Modelo de Grafo (Neo4j)

O Protótipo B implementou o modelo de Grafo de Propriedades Rotuladas (*Labeled Property Graph* - LPG). As entidades relacionais fortes foram convertidas em Nós (:Cliente, :Pedido, :Produto, :Vendedor). A divergência estrutural central ocorreu no tratamento da entidade associativa *itens_pedido*, que foi suprimida como nó intermediário e convertida fisicamente na aresta direcional [:CONTEM].

Atributos transacionais quantitativos, como preço e quantidade, foram embarcados como propriedades internas deste relacionamento. Essa modelagem encurtou os caminhos de busca e potencializou o princípio da Adjacência Livre de Índice, permitindo que, após a localização do nó inicial, a consulta percorresse diretamente os relacionamentos armazenados no grafo, com custo associado principalmente à quantidade de conexões efetivamente exploradas.

Figura 8 - Modelo de grafo do Protótipo B (Neo4j)



Fonte: Autoria própria (2025)

4.2 Arquitetura do Motor de Benchmark e Captura de Discrepâncias

A orquestração do experimento foi delegada a um motor desenvolvido em Python, parametrizado para isolar o *hardware* e capturar o comportamento em tempo real das ferramentas.

Uma premissa imperativa do motor foi a **amostragem dinâmica de chaves a cada iteração**. Diferentemente de testes estáticos que pesquisam a mesma entidade repetidas vezes, o sistema sorteava aleatoriamente clientes e produtos ativos na base antes de cada ciclo. Esse mecanismo foi projetado especificamente para impedir que os bancos de dados viciassem o plano de execução no *cache* da RAM.

A amostragem dinâmica foi a responsável direta por isolar e explicar o porquê das severas **discrepâncias de desempenho observadas entre execuções do mesmo teste**:

- **A Discrepância no Modelo Relacional (GEQO e Colapso):** Ao sortear clientes com redes profundas, o otimizador relacional ultrapassa seu limite de planejamento hierárquico, recorrendo ao Otimizador de Consulta Genética (GEQO). A tentativa de resolver produtos cartesianos massivos com caminhos esparsos gera flutuações extremas, indo de poucos milissegundos à interrupção por *timeout* (esgotamento de recursos).
- **A Discrepância no Modelo de Grafos (Super-Nó):** O sorteio dinâmico revelou o impacto topológico de itens altamente populares. Quando a consulta incide

sobre um "Super-Nó" (um produto comprado por dezenas de milhares de usuários), o tempo de travessia no Neo4j aumenta pontualmente devido ao esforço físico de varrer milhares de ponteiros, elevando a média e o desvio-padrão entre as baterias, embora o banco sobreviva à execução sem estourar a memória estrutural.

Para proteger a integridade da aplicação contra o colapso relacional em testes extremos, foi implementada uma degradação graciosa com *Statement Timeout* de 120 segundos.

4.3 Protótipos das Consultas

Os cenários de *benchmark* foram elaborados em escala crescente de complexidade e profundidade topológica.

Consulta Q1 (Filtragem Colaborativa Simples)

- **Lógica:** Dado um produto alvo, encontra quais outros itens foram comprados pelos mesmos clientes que o adquiriram.
- **SQL:** Exige cinco autojunções limitadas a dois saltos lógicos a partir do produto.
- **Cypher:**

```
MATCH (p1:Produto) <-[:CONTEM]-(:Pedido) <-[:FEZ_PEDIDO]-(:Cliente) <-[:FEZ_PEDIDO]->(:Pedido) <-[:CONTEM]->(p2:Produto).
```

Consulta Q2 (Análise de Cesta de Compras - Coocorrência)

- **Lógica:** Identifica produtos frequentemente adquiridos no exato mesmo pedido, cruzando a entidade associativa consigo mesma.
- **SQL:** Junção mais curta do escopo, cruzando `itens_pedido` diretamente.
- **Cypher:** Varredura imediata de nós adjacentes a partir do nó `:Pedido`.

Consulta Q3 (Recomendação Híbrida - Perfil e Geografia)

- **Lógica:** Busca vizinhos demográficos (mesma cidade) e recomenda os itens adquiridos por eles, restritos às categorias de interesse histórico do cliente alvo.
- **Desafio Arquitetural:** Exige alto tráfego em memória para agregação de grandes conjuntos geográficos e comparação de metadados antes da travessia lógica, impactando o desempenho nativo dos grafos.

Série Q4 (Teste de Estresse Escalonado - A Explosão Combinatória)

- **Lógica:** Mapeia a rede de influência oculta do cliente por expansão de vizinhança. Escala de Q4_G1 (compras diretas) até Q4_G4 (vizinhança de terceiro nível).
- **SQL:** A progressão até Q4_G4 exige 14 `JOINS` sucessivos, resultando em hipertrofia do produto cartesiano e colapso computacional algorítmico do PostgreSQL.
- **Cypher:** Representação sequencial elegante de conexões: `(Cliente)-...-(Outro1)-...-(Outro2)-...-(Outro3)-...->(Produto)`.

Consulta Q5 (O Caminho Mais Curto - Disparidade de Expressividade)

- **Lógica:** Calcula o menor número de saltos que conecta o Cliente A ao Cliente B com base em padrões de consumo.
- **SQL:** A baixa aderência do modelo relacional à descoberta de redes desconhecidas força o uso de Expressões de Tabela Comum Recursivas (`WITH RECURSIVE`), resultando em latência impraticável e código denso.
- **Cypher:** Solução nativa através do algoritmo semântico interno `shortestPath()`, alcançando resolução assintótica altamente eficiente.

5 ANÁLISE DOS RESULTADOS E DISCUSSÃO

Este capítulo dedica-se à apresentação, interpretação e discussão das métricas de latência coletadas pelo motor de *benchmark*. Os resultados foram consolidados a partir da execução dinâmica das consultas nos protótipos relacional (PostgreSQL) e de grafos (Neo4j), com o objetivo de responder ao problema central da pesquisa: identificar o ganho real de desempenho e expressividade arquitetural na transição entre esses paradigmas.

Os testes foram executados com uma amostragem randômica a cada iteração, garantindo que o tempo de resposta refletisse o verdadeiro custo de processamento algorítmico (uso de CPU e memória) e não a simples leitura de um plano de execução armazenado em *cache*. As latências analisadas referem-se estritamente ao estado de *warm cache* (após 5 ciclos de aquecimento descartados), compondo uma amostra estabilizada para cada cenário.

5.1 Desempenho em Consultas de Baixa Profundidade

O primeiro bloco do experimento (Consultas Q1, Q2 e Q3) objetivou avaliar a capacidade dos motores em lidar com filtragens colaborativas clássicas, caracterizadas por uma profundidade de rede baixa (geralmente limitadas a um ou dois saltos semânticos).

Os dados quantitativos revelaram uma superioridade expressiva do modelo relacional nos cenários de baixa profundidade avaliados neste domínio. A Tabela 1 consolida as médias de tempo de resposta em milissegundos (ms) para este bloco.

Tabela 1 Tempo de Resposta (ms) para Consultas de Baixa Profundidade

Consulta	Regra de Negócio	PostgreSQL (Média ms)	Neo4j (Média ms)	Vantagem
Q1	Filtragem Colaborativa Simples	2,12	2,97	PostgreSQL
Q2	Coocorrência (Cesta de Compras)	0,35	2,90	PostgreSQL
Q3	Recomendação Híbrida (Perfil)	34,17	22,17	Neo4j

Fonte: Autoria própria (2026)

A consulta **Q2**, focada na detecção de afinidade em cestas de compras, representou o cenário de maior disparidade a favor do PostgreSQL, executando em 0,35 ms contra 2,90 ms do Neo4j. Esse fenômeno valida a eficiência extrema do algoritmo de *Hash Join* em bancos de dados relacionais quando operam sobre conjuntos de dados que cabem inteiramente na memória RAM primária do servidor. Como a consulta Q2 cruza a tabela `itens_pedido` consigo mesma e faz uso intensivo dos índices B-Tree, a resolução do produto matricial ocorre sem a necessidade de paginação em disco.

A exceção a essa regra de superioridade ocorreu na consulta **Q3** (Recomendação Híbrida), onde o Neo4j apresentou uma média mais ágil (22,17 ms contra 34,17 ms do PostgreSQL). Contudo, ambas as plataformas registraram seus piores tempos dentro deste bloco analítico. Essa latência elevada deve-se ao perfil da consulta, que exige o cruzamento não apenas de identificadores numéricos, mas o agrupamento massivo de entidades pela categoria de região (`customer_city`) e avaliações de pertencimento a conjuntos de metadados dispersos. Esse comportamento comprova que agrupamentos baseados em propriedades de texto aberto não são o cenário ideal para nenhum dos dois paradigmas avaliados

5.2 O Teste de Estresse e a Explosão Combinatória

A divergência arquitetural definitiva entre os sistemas foi mapeada durante a execução da **Série Q4**, o teste de estresse escalonado projetado para forçar a expansão de vizinhança. O objetivo deste bloco foi localizar matematicamente o ponto de colapso do modelo tabular. A Tabela 2 apresenta o impacto exponencial do aumento da profundidade relacional.

Tabela 2 - Escalonamento de Profundidade e Degradação de Performance (ms)

Grau de Profundidade	Lógica de Vizinhança	Qtd. de JOINS (SQL)	PostgreSQL (Média ms)	Neo4j (Média ms)
Q4_G1	Compras Diretas (1 salto)	2 JOINS	0,44	1,99
Q4_G2	Vizinhança de 1º Nível	6 JOINS	2,78	3,28
Q4_G3	Vizinhança de 2º Nível	10 JOINS	273,28	8,27

Grau de Profundidade	Lógica de Vizinhaça	Qtd. de JOINS (SQL)	PostgreSQL (Média ms)	Neo4j (Média ms)
Q4_G4	Vizinhaça de 3º Nível	14 JOINS	[TIMEOUT]	6731,91

Fonte: Autoria própria (2026)

Os resultados empíricos expuseram de forma agressiva a limitação arquitetural do modelo relacional (PostgreSQL) ao lidar com redes estendidas. Enquanto a transição do Grau 1 para o Grau 2 manteve os tempos nivelados, a inserção do Grau 3 (10 *JOINS* aninhados) forçou a média do PostgreSQL a saltar de 2,78 ms para 273,28 ms. O colapso total do SGBD ocorreu no Grau 4 (Q4_G4). Ao tentar resolver 14 *JOINS* simultâneos, o otimizador relacional ultrapassou seu limite computacional, esgotando o limite de 120 segundos da aplicação (*Timeout*) e falhando em entregar o resultado devido ao custo exponencial ($O(N^k)$) do Produto Cartesiano gerado na memória.

Em contrapartida, a **Adjacência Livre de Índice** do Neo4j demonstrou sua resiliência estrutural. A latência cresceu linearmente de forma controlada nos primeiros três graus (de 1,99 ms para 8,27 ms). O tempo mais elevado registrado pelo Neo4j na Q4_G4 (6,7 segundos) ilustra o mapeamento prático do "Problema do Super-Nó" (*Supernode Problem*). Ao sortear randômicamente clientes conectados a produtos atípicos de altíssima popularidade, o motor de travessia do Cypher precisou varrer milhares de arestas fisicamente no disco. O dado crítico, no entanto, é a **sobrevivência sistêmica**: o banco de dados de grafos resolveu o problema e devolveu a requisição, enquanto a arquitetura relacional simplesmente deixou de operar no mesmo hardware.

5.3 A Descoberta de Caminhos (Shortest Path) e Expressividade

O último cenário analítico (Q5) afastou-se da recomendação tradicional para explorar a capacidade dos paradigmas em resolver problemas de descoberta de rede. A regra de negócio demandava o cálculo do menor caminho logístico (número de saltos) conectando dois clientes aleatórios. A Tabela 3 evidencia o abismo de *performance* nesta categoria algorítmica.

Tabela 3 - Tempo de Resposta (ms) para Descoberta de Caminho (Q5)

Consulta	Lógica Algorítmica	PostgreSQL (Média ms)	Neo4j (Média ms)	Vantagem
Q5	Shortest Path (Graus de Separação)	1595,35	9,80	Neo4j (162x mais rápido)

Fonte: Autoria própria (2026)

A latência média de 1,5 segundos (1595 ms) do PostgreSQL atesta que o modelo relacional é estruturalmente ineficiente para inferir topologias desconhecidas. Como o SQL foi concebido para retornar dados a partir de rotas explicitamente declaradas pelo desenvolvedor, a busca por caminhos indefinidos obriga o uso de Expressões de Tabela Comum Recursivas (`WITH RECURSIVE`). Esse recurso gera iterações exaustivas na memória RAM, testando caminhos cegos de forma iterativa, o que penaliza severamente o desempenho da CPU.

O Neo4j, por sua vez, demonstrou sua superioridade semântica e algorítmica ao solucionar o mesmo problema em 9,80 milissegundos. A linguagem Cypher expõe a função nativa `shortestPath()`, que utiliza estratégias internas de travessia em largura, incluindo busca bidirecional em determinados cenários, para percorrer os relacionamentos armazenados no grafo e encontrar a conexão entre os nós analisados. Esse resultado indica que o banco de grafos não apenas simplifica a escrita e a manutenção do código por parte da equipe de engenharia, mas também apresentou desempenho muito superior ao modelo relacional no cenário de descoberta de caminhos avaliado neste experimento (NEO4J, 2026).

6 CONSIDERAÇÕES FINAIS

O presente trabalho propôs-se a investigar e quantificar as vantagens e desvantagens arquiteturais na adoção de Bancos de Dados Orientados a Grafos em detrimento dos tradicionais Modelos Relacionais, focando especificamente no processamento de sistemas de recomendação para o comércio eletrônico. Por meio do desenvolvimento de um motor de *benchmark* rigoroso e isolado, operando sobre o conjunto de dados reais da Olist, foi possível mapear o ponto exato de inflexão estrutural entre as duas tecnologias.

Os resultados empíricos atingiram com êxito os objetivos delineados. Comprovou-se que a premissa de que "grafos são sempre mais rápidos para relacionamentos" constitui uma generalização imprecisa. Para recomendações rasas e filtragens colaborativas clássicas (como análise de cestas de compras) operando sobre *datasets* de volumetria moderada — capazes de serem integralmente armazenados na memória RAM primária —, a indexação B-Tree do PostgreSQL superou o Neo4j com margens expressivas, operando na casa das frações de milissegundo.

Contudo, a pesquisa evidenciou o limite crítico da arquitetura relacional perante a explosão combinatória. Ao submeter os protótipos a um teste de estresse escalonado simulando redes de vizinhança profunda (a partir do terceiro nível de separação), o mecanismo de junção do PostgreSQL entrou em colapso. O otimizador genético perdeu a capacidade de traçar planos eficientes, culminando em esgotamento de *hardware* (*timeout* de execução). Em contraste agudo, o princípio da Adjacência Livre de Índice garantiu a resiliência do Neo4j, que manteve complexidade assintótica linear e estabilidade de execução extrema durante as travessias profundas e na aplicação de algoritmos nativos de caminhos mais curtos (*Shortest Path*).

Conclui-se que arquitetos de *software* não devem migrar para soluções de grafos visando otimizar consultas analíticas isoladas de baixa complexidade. A adoção do Neo4j justifica-se especialmente quando o domínio da aplicação exige escalabilidade para consultas hierárquicas não determinísticas, inferência estrutural e algoritmos de travessia acima de dois saltos lógicos, cenários nos quais o modelo tabular relacional tende a apresentar degradação significativa.

Como trabalhos futuros, sugere-se a expansão deste *benchmark* para ambientes distribuídos em nuvem (*clusters*), visando avaliar o impacto da latência de

rede no protocolo Bolt do Neo4j. Adicionalmente, recomenda-se a aplicação das mesmas consultas em bancos de dados massivos — na escala de centenas de milhões de registros —, a fim de mapear o desempenho do PostgreSQL quando o índice B-Tree excede o tamanho disponível na memória volátil, forçando o gargalo físico de paginação de disco.

REFERÊNCIAS

ELMASRI, Ramez; NAVATHE, Shamkant B. **Sistemas de banco de dados**. 6. ed. São Paulo: Pearson Addison-Wesley, 2011.

SADALAGE, Pramod J.; FOWLER, Martin. **NoSQL distilled: a brief guide to the emerging world of polyglot persistence**. Boston: Addison-Wesley, 2012.

CODD, Edgar F. A relational model of data for large shared data banks. **Communications of the ACM**, New York, v. 13, n. 6, p. 377-387, 1970. DOI: 10.1145/362384.362685.

POSTGRESQL GLOBAL DEVELOPMENT GROUP. **PostgreSQL documentation: index types**. [S. l.]: PostgreSQL Global Development Group, 2026a. Disponível em: <https://www.postgresql.org/docs/current/indexes-types.html>. Acesso em: 6 jun. 2026.

POSTGRESQL GLOBAL DEVELOPMENT GROUP. **PostgreSQL documentation: B-tree indexes**. [S. l.]: PostgreSQL Global Development Group, 2026b. Disponível em: <https://www.postgresql.org/docs/current/btree.html>. Acesso em: 6 jun. 2026.

POSTGRESQL GLOBAL DEVELOPMENT GROUP. PostgreSQL documentation: genetic query optimization in PostgreSQL. [S. l.]: PostgreSQL Global Development Group, 2026c. Disponível em: <https://www.postgresql.org/docs/current/geqo-pg-intro.html>. Acesso em: 6 jun. 2026.

ROBINSON, Ian; WEBBER, Jim; EIFREM, Emil. **Graph databases: new opportunities for connected data**. 2. ed. Sebastopol: O'Reilly Media, 2015.

RICCI, Francesco; ROKACH, Lior; SHAPIRA, Bracha; KANTOR, Paul B. (ed.). **Recommender systems handbook**. Boston: Springer, 2011. DOI: 10.1007/978-0-387-85820-3.

AGGARWAL, Charu C. **Recommender systems: the textbook**. Cham: Springer, 2016. DOI: 10.1007/978-3-319-29659-3.

OLIST. **Brazilian E-Commerce Public Dataset by Olist**. Kaggle, [s. d.]. Disponível em: <https://www.kaggle.com/datasets/olistbr/brazilian-ecommerce>. Acesso em: 6 jun. 2026.

NEO4J. **Shortest paths: Cypher Manual**. [S. l.]: Neo4j, 2026. Disponível em: <https://neo4j.com/docs/cypher-manual/current/patterns/shortest-paths/>. Acesso em: 6 jun. 2026.